

# **Q-Sec: Quantum Secure Chat**

Project Team

|               |          |
|---------------|----------|
| Basim Ahmad   | 20I-2456 |
| Muhammad Maaz | 20I-0837 |
| Sarib Aftab   | 20I-2706 |

Session 2020-2025

Supervised by

**Mr. Jawad Hassan**



**Department of Computer Science**

**National University of Computer and Emerging Sciences  
Islamabad, Pakistan**

**June, 2024**

# Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                | <b>1</b>  |
| 1.1      | Problem Statement . . . . .                        | 1         |
| 1.2      | Scope . . . . .                                    | 2         |
| 1.3      | Modules . . . . .                                  | 2         |
| 1.3.1    | Module 1: Quantum Key Distribution (QKD) . . . . . | 3         |
| 1.3.2    | Module 2: AES Encryption . . . . .                 | 3         |
| 1.3.3    | Module 3: Real-Time Communication . . . . .        | 3         |
| 1.3.4    | Module 4: User Authentication . . . . .            | 3         |
| 1.3.5    | Module 5: Front-end User Interface . . . . .       | 3         |
| 1.3.6    | Module 6: Back-end Server . . . . .                | 4         |
| 1.3.7    | Module 7: Quantum Secret Sharing (QSS) . . . . .   | 4         |
| 1.4      | User Classes and Characteristics . . . . .         | 4         |
| <b>2</b> | <b>Project Requirements</b>                        | <b>7</b>  |
| 2.1      | Use Case Diagram . . . . .                         | 7         |
| 2.1.1    | Detailed Use Cases . . . . .                       | 9         |
| 2.2      | Functional Requirements . . . . .                  | 16        |
| 2.2.1    | Module 1: Quantum Key Distribution (QKD) . . . . . | 16        |
| 2.2.2    | Module 2: AES Encryption . . . . .                 | 16        |
| 2.2.3    | Module 3: Real-Time Communication . . . . .        | 17        |
| 2.2.4    | Module 4: User Authentication . . . . .            | 17        |
| 2.2.5    | Module 5: Front-end User Interface . . . . .       | 17        |
| 2.2.6    | Module 6: Back-end Server . . . . .                | 18        |
| 2.2.7    | Module 7: Quantum Secret Sharing (QSS) . . . . .   | 18        |
| 2.3      | Non-Functional Requirements . . . . .              | 19        |
| 2.3.1    | Security . . . . .                                 | 19        |
| 2.3.2    | Reliability . . . . .                              | 19        |
| 2.3.3    | Performance . . . . .                              | 19        |
| 2.3.4    | Usability . . . . .                                | 20        |
| 2.4      | Domain Model . . . . .                             | 20        |
| <b>3</b> | <b>System Overview</b>                             | <b>23</b> |
| 3.0.1    | Functionality . . . . .                            | 23        |

|         |                                      |           |
|---------|--------------------------------------|-----------|
| 3.0.2   | Context and Design . . . . .         | 24        |
| 3.0.3   | Key Modules of the System: . . . . . | 24        |
| 3.0.4   | Background Information . . . . .     | 25        |
| 3.0.5   | Design of the System . . . . .       | 25        |
| 3.1     | Architectural Design . . . . .       | 25        |
| 3.2     | Design Models . . . . .              | 27        |
| 3.2.1   | Activity Diagram . . . . .           | 27        |
| 3.2.2   | Sequence Diagram . . . . .           | 28        |
| 3.2.3   | State Transition Diagram . . . . .   | 28        |
| 3.3     | Data Design . . . . .                | 30        |
| 3.3.1   | Data Entities . . . . .              | 30        |
| 3.3.1.1 | Users . . . . .                      | 30        |
| 3.3.1.2 | Messages . . . . .                   | 31        |
| 3.3.1.3 | Quantum Keys . . . . .               | 31        |
| 3.3.1.4 | Encryption Keys . . . . .            | 32        |
| 3.3.1.5 | System Logs . . . . .                | 32        |
| 3.3.1.6 | Groups . . . . .                     | 33        |
| 3.3.2   | Data Flow and Processing . . . . .   | 33        |
| 3.3.3   | Data Storage Technologies . . . . .  | 34        |
| 3.3.4   | Conclusion . . . . .                 | 34        |
|         | <b>References</b>                    | <b>35</b> |

# List of Figures

|     |                                              |    |
|-----|----------------------------------------------|----|
| 2.1 | Use Case Diagram for Q-Sec . . . . .         | 8  |
| 2.2 | Domain Model Diagram for Q-Sec . . . . .     | 21 |
| 3.1 | Architecture Diagram for Q-Sec . . . . .     | 26 |
| 3.2 | Activity Diagram for Q-Sec . . . . .         | 27 |
| 3.3 | Sequence Diagram for Q-Sec . . . . .         | 28 |
| 3.4 | State Transition Diagram for Q-Sec . . . . . | 29 |

# List of Tables

|     |                                                       |    |
|-----|-------------------------------------------------------|----|
| 1.1 | User Classes and Characteristics for Q-Sec . . . . .  | 5  |
| 2.1 | Use Case 1: User Registration . . . . .               | 9  |
| 2.2 | Use Case 2: User Authentication . . . . .             | 10 |
| 2.3 | Use Case 3: Send Encrypted Message . . . . .          | 11 |
| 2.4 | Use Case 4: Receive Encrypted Message . . . . .       | 12 |
| 2.5 | Use Case 5: Quantum Key Distribution (QKD) . . . . .  | 13 |
| 2.6 | Use Case 6: Send Group Encrypted Message . . . . .    | 14 |
| 2.7 | Use Case 7: Receive Group Encrypted Message . . . . . | 15 |

# Chapter 1

## Introduction

This project proposal document demonstrates the design and implementation of Q-Sec which is a chat application with the use of quantum cryptographic and advanced encryption standard encryption systems. Thus, in the context of today's contacts and computerization, the communication security is becoming more of a problem because of increased cyber attacks and the future threat of having to deal with quantum computing. Conventional encryption techniques, such as RSA, are also at risk to be hacked by quantum computers mainly employing the Shor's Algorithm. To overcome this, Q-Sec combines QKD based on the BB84 protocol for a secure exchange of keys and Advanced Encryption Standard (AES) to protect message content. This consolidated solution is for achieving the future-proof secure communication. The background of the system is centered on the issue of the lack of adequate levels of data security to support communication channels that are highly sensitive as seen in the financial sector, health care, and most importantly the government. It is for this reason that Q-Sec is under development since it aims at tackling many risks that new systems such as the quantum computers pose to current cryptography. [4]

### 1.1 Problem Statement

Communications networks face threats from cyber-attacks, and with the coming of age of quantum computing, classical cryptography is bound to become penetrable. Most of today's chat applications such as WhatsApp and Signal use either RSA or ECC that are vulnerable to being decrypted by quantum computers using Shor's Algorithm [3]. This forms a big security loophole especially in sectors where communication most often has to be secure such as healthcare, the banking sector, government, and defense. [1]

The issue that Q-Sec solves is how real-time communication can be protected from future quantum threats. The main problem is to make the exchanged messages between users

secure and invulnerable to interception by an opponent who has access to quantum computers. This project aims to develop a cryptographic system that will use QKD for key distribution and AES for message encryption to ensure that communication will be safe now and in the future, when quantum computers become more powerful. [2]

## 1.2 Scope

The purpose of the Q-Sec project is to create a quantum-secure chat application that employs Quantum Key Distribution with the BB84 protocol and the Advanced Encryption Standard (AES) Cipher for encrypting messages exchanged by the users. The application will include real-time messaging with end-to-end encryption using keys generated by a quantum system. The project focuses on developing an easy-to-understand graphical interface for the web where security protocols are implemented without the user needing to understand the encryption algorithms used.

Some functionalities that will be implemented in Q-Sec include quantum key exchange, AES message encryption to enhance the security of transmitting messages via the internet, real-time message transmission using secure socket technology, and user authentication to ensure data is only accessed by authorized individuals. Additionally, eavesdropping prevention and error-checking features will be integrated into the QKD system. The backend will be built using Node.js, which will manage keys, encrypt/decrypt messages, and interact with users. Frontend development will use client-side tools like HTML5, CSS3, and JavaScript to design an interactive client interface.

Although the system will be designed as a web-based application, it has the potential for future development on mobile platforms. The scope also involves developing secure storage where messages and user information are stored in an encrypted format, with an assurance of security even when the databases are idle. Q-Sec aims to combine quantum cryptography with traditional encryption methods to ensure secure, scalable communication solutions that can defend against current and future threats.

## 1.3 Modules

The Q-Sec project consists of several modules that are crucial for ensuring the security, functionality, and usability of the application. Below are the key modules:

### **1.3.1 Module 1: Quantum Key Distribution (QKD)**

This module handles the secure exchange of cryptographic keys using the BB84 protocol. QKD ensures that encryption keys cannot be intercepted during transmission.

1. Implementation of the BB84 protocol for key exchange.
2. Eavesdropping detection and quantum key verification.

### **1.3.2 Module 2: AES Encryption**

This module is responsible for encrypting and decrypting the messages exchanged between users using AES encryption, utilizing the keys generated by the QKD module.

1. Encrypt messages using AES-256.
2. Decrypt incoming messages using AES-256.

### **1.3.3 Module 3: Real-Time Communication**

This module facilitates real-time message transmission between users via secure sockets, ensuring low-latency communication with end-to-end encryption.

1. Secure real-time messaging using Socket.io.
2. Secure sockets for encrypted communication.

### **1.3.4 Module 4: User Authentication**

This module ensures that only authenticated users can access the chat system. Authentication methods will include secure login mechanisms and user verification.

1. Secure login system.
2. Multi-factor authentication for enhanced security.

### **1.3.5 Module 5: Front-end User Interface**

The frontend module will create a user-friendly and responsive web interface for users to exchange messages securely without needing to understand the underlying cryptographic processes.



1. Web-based interface using React.js with HTML5, CSS3, and JavaScript.
2. Responsive design for ease of use.

### **1.3.6 Module 6: Back-end Server**

This module is responsible for managing application logic, client requests, and facilitating real-time communication.

1. Manages application logic.
2. Handles incoming client requests and responses.
3. Manages real-time communication.

### **1.3.7 Module 7: Quantum Secret Sharing (QSS)**

This module ensures secure group communication using Quantum Secret Sharing (QSS). QSS allows multiple parties (users) to share a quantum key, ensuring that any message sent to the group can only be decrypted when a quorum of users (the intended group) is present.

1. Implementation of QSS for group key distribution.
2. Securely sharing a single quantum key among group members.
3. Ensuring message decryption requires participation from all group members.

## **1.4 User Classes and Characteristics**

The various user classes for Q-Sec are as follows:

| User Class                    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| End Users (Chat Users)        | End Users include individuals or organizations using Q-Sec for secure real-time communication. These users rely on Q-Sec's encryption and key distribution protocols to ensure confidentiality. The primary expectation is for seamless, secure messaging with minimal need for technical knowledge about encryption mechanisms like QKD or AES. They will interact through an intuitive user interface for sending and receiving encrypted messages. Users are authenticated via secure login methods, including multi-factor authentication (MFA).                                                                                      |
| Administrative Users (Admins) | Admins oversee the security and performance of the Q-Sec system. They manage user authentication, quantum key exchanges, and real-time messaging. Admins also monitor the system for any eavesdropping attempts and handle eavesdropping detection alerts, key management, and security breaches. They are responsible for log monitoring, performing regular security audits, and ensuring compliance with the latest cryptographic standards. Admins must have a strong background in cryptography, particularly QKD and AES, as well as system maintenance.                                                                            |
| Developers/System Maintainers | Developers ensure that the QKD module, AES encryption, and real-time communication systems are running efficiently and securely. They handle updates to the backend infrastructure, which includes Node.js, Socket.io, and MongoDB, and implement new features. Developers must be proficient in secure communication protocols and maintain the system's cryptographic capabilities. Their responsibilities also include integrating the QKD protocol (BB84) with AES-256 for encryption and managing secure data storage. System performance tuning, patching vulnerabilities, and responding to security incidents are also key roles. |

Table 1.1: User Classes and Characteristics for Q-Sec



# **Chapter 2**

## **Project Requirements**

This chapter describes the functional and non-functional requirements of the project.

### **2.1 Use Case Diagram**

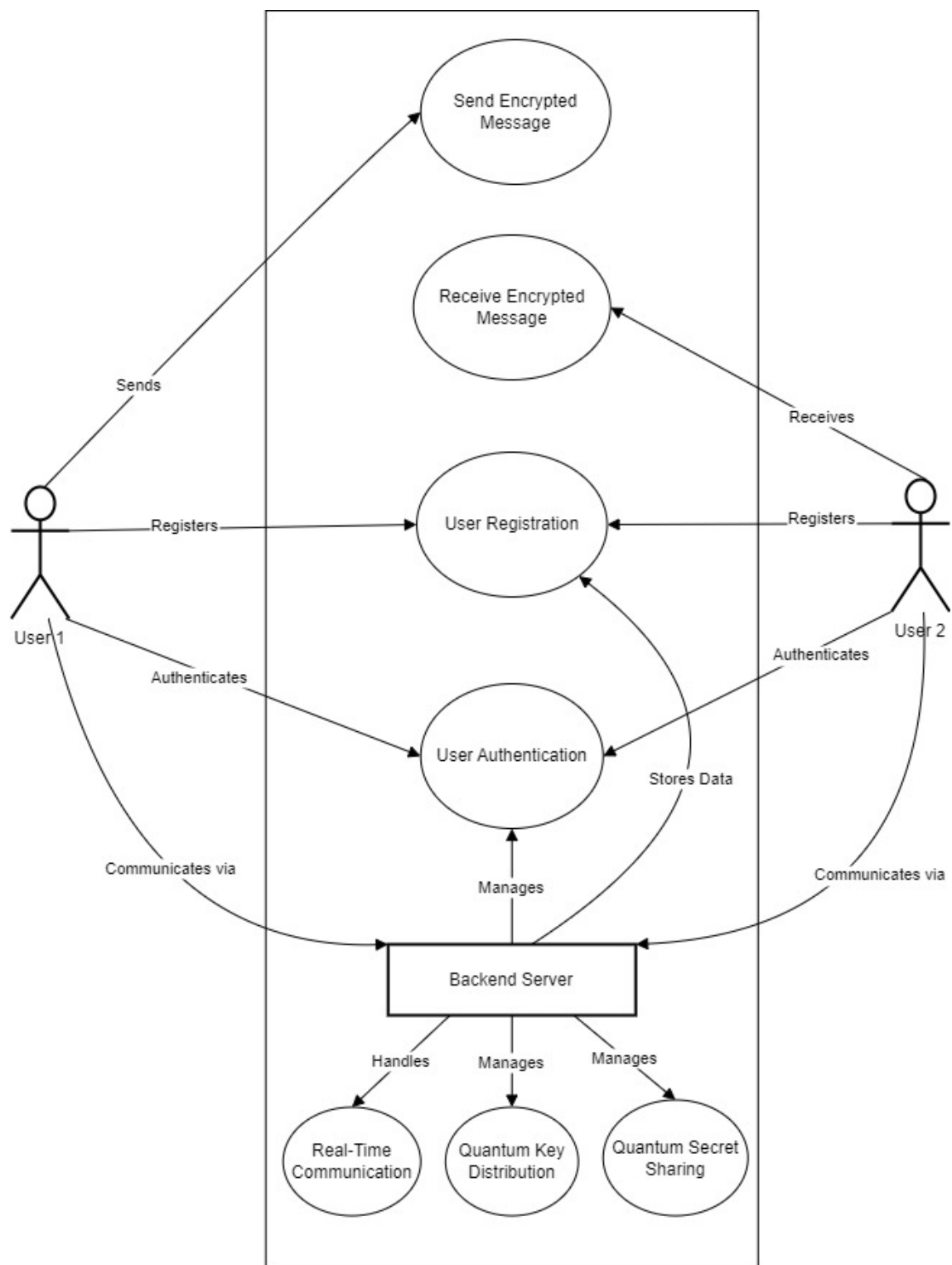


Figure 2.1: Use Case Diagram for Q-Sec

### 2.1.1 Detailed Use Cases

The following are the detailed use cases for Q-Sec.

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-01                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Use Case Name</b>     | User Registration                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Actors</b>            | User 1, Backend Server                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Preconditions</b>     | The user has not yet registered for the system and is accessing the registration page through the React.js interface.                                                                                                                                                                                                                                                                                                         |
| <b>Description</b>       | The user provides required information to create an account in the Q-Sec system. The backend server, built on Node.js, manages the registration process, validates user data, and stores user credentials securely in MongoDB. Communication between the user and the server is secured using SSL/TLS.                                                                                                                        |
| <b>Postconditions</b>    | The user account is created and stored in MongoDB. The user can now log in and use the system.                                                                                                                                                                                                                                                                                                                                |
| <b>Basic Flow</b>        | <ol style="list-style-type: none"> <li>1. User 1 accesses the registration page via the React.js interface.</li> <li>2. User 1 enters required information (e.g., username, password, email).</li> <li>3. System validates the information via the backend server (Node.js).</li> <li>4. The backend server stores the user credentials in MongoDB.</li> <li>5. System sends a confirmation email (if applicable).</li> </ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"> <li>1. If the username is already taken, the system prompts the user to choose a different username.</li> <li>2. If the email format is invalid, the system prompts the user to enter a valid email.</li> </ol>                                                                                                                                                                            |
| <b>Exceptions</b>        | <ol style="list-style-type: none"> <li>1. Registration system is down or unreachable.</li> <li>2. Invalid information provided by the user (e.g., weak password).</li> </ol>                                                                                                                                                                                                                                                  |

Table 2.1: Use Case 1: User Registration

|                          |                                                                                                                                                                                                                                                                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-02                                                                                                                                                                                                                                                                                                                                         |
| <b>Use Case Name</b>     | User Authentication                                                                                                                                                                                                                                                                                                                           |
| <b>Actors</b>            | User 1, User 2, Backend Server                                                                                                                                                                                                                                                                                                                |
| <b>Preconditions</b>     | User has valid credentials (username and password) or multi-factor authentication is enabled.                                                                                                                                                                                                                                                 |
| <b>Description</b>       | The user logs into the system using secure login mechanisms and multi-factor authentication (if enabled). The backend server manages authentication using Node.js, and secure communication is established with SSL/TLS.                                                                                                                      |
| <b>Postconditions</b>    | The user is authenticated and granted access to the system. The user's data is stored securely in MongoDB.                                                                                                                                                                                                                                    |
| <b>Basic Flow</b>        | <ol style="list-style-type: none"><li>1. User navigates to the login page in the React.js interface.</li><li>2. User enters their username and password.</li><li>3. System verifies the credentials via Node.js backend.</li><li>4. Upon successful verification, the user is logged in and a session is established using SSL/TLS.</li></ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"><li>1. If authentication fails, the user is prompted to retry.</li><li>2. If multi-factor authentication is enabled, the user must also verify through the second factor.</li></ol>                                                                                                                         |
| <b>Exceptions</b>        | <ol style="list-style-type: none"><li>1. Account lockout after multiple failed attempts.</li><li>2. Server unavailable for authentication.</li></ol>                                                                                                                                                                                          |

Table 2.2: Use Case 2: User Authentication

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-03                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Use Case Name</b>     | Send Encrypted Message                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Actors</b>            | User 1, Backend Server                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Preconditions</b>     | User 1 is authenticated, and a quantum key has been exchanged using the BB84 protocol via the QKD module.                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Description</b>       | User 1 sends a message that is encrypted using AES-256 with a quantum key generated by the QKD module. The backend server (built using Node.js) securely transmits the encrypted message to User 2 using Socket.io and SSL/TLS for secure communication.                                                                                                                                                                                                                                   |
| <b>Postconditions</b>    | The encrypted message is transmitted to User 2 via the backend server, and the message is securely stored in MongoDB.                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Basic Flow</b>        | <ol style="list-style-type: none"> <li>1. User 1 composes a message in the React.js web interface.</li> <li>2. System retrieves the quantum key generated by the QKD module.</li> <li>3. The message is encrypted using AES-256.</li> <li>4. The encrypted message is sent to the backend server via Socket.io.</li> <li>5. Backend server transmits the encrypted message securely to User 2 over SSL/TLS.</li> <li>6. The message is stored in MongoDB in its encrypted form.</li> </ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"> <li>1. If the key retrieval fails, the system notifies the user and retries.</li> <li>2. If the recipient (User 2) is offline, the message is queued securely for later delivery.</li> </ol>                                                                                                                                                                                                                                                            |
| <b>Exceptions</b>        | <ol style="list-style-type: none"> <li>1. Network failure during message transmission.</li> <li>2. Failure of the QKD module to generate or retrieve a key.</li> </ol>                                                                                                                                                                                                                                                                                                                     |

Table 2.3: Use Case 3: Send Encrypted Message



|                          |                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-04                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Use Case Name</b>     | Receive Encrypted Message                                                                                                                                                                                                                                                                                                                                                |
| <b>Actors</b>            | User 2, Backend Server                                                                                                                                                                                                                                                                                                                                                   |
| <b>Preconditions</b>     | User 2 is authenticated and has established a quantum key with User 1 via the QKD module.                                                                                                                                                                                                                                                                                |
| <b>Description</b>       | User 2 receives the encrypted message from the backend server via Socket.io. The system retrieves the quantum key and decrypts the message using AES-256. Communication is secured by SSL/TLS, and the message is displayed on the React.js web interface.                                                                                                               |
| <b>Postconditions</b>    | 1. The message is successfully decrypted and displayed to User 2.<br>2. The encrypted message is stored securely in MongoDB.                                                                                                                                                                                                                                             |
| <b>Basic Flow</b>        | <ol style="list-style-type: none"><li>1. User 2 receives a notification in the React.js interface about the incoming encrypted message.</li><li>2. System retrieves the associated quantum key from the QKD module.</li><li>3. The message is decrypted using AES-256.</li><li>4. The decrypted message is displayed to User 2 via the React.js web interface.</li></ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"><li>1. If the key retrieval fails, the system notifies the user and retries.</li><li>2. If the message is corrupted, the system requests a re-transmission.</li></ol>                                                                                                                                                                  |
| <b>Exceptions</b>        | <ol style="list-style-type: none"><li>1. Message tampering detected by the system.</li><li>2. User 2 is not authenticated.</li></ol>                                                                                                                                                                                                                                     |

Table 2.4: Use Case 4: Receive Encrypted Message

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-05                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Use Case Name</b>     | Quantum Key Distribution (QKD)                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Actors</b>            | User 1, User 2, Backend Server                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Preconditions</b>     | Both users are authenticated, and the system is ready to establish a secure communication channel using Node.js and Socket.io.                                                                                                                                                                                                                                                                                                                     |
| <b>Description</b>       | The QKD module generates and distributes quantum encryption keys between User 1 and User 2 through the backend server. Communication is secured via SSL/TLS, and eavesdropping detection is performed during the key exchange.                                                                                                                                                                                                                     |
| <b>Postconditions</b>    | A secure encryption key is exchanged between the two users, ready for use in AES-256 encryption. The keys are stored securely in MongoDB for future use.                                                                                                                                                                                                                                                                                           |
| <b>Basic Flow</b>        | <ol style="list-style-type: none"> <li>1. The system initializes the BB84 protocol for key distribution.</li> <li>2. Quantum keys are generated by the QKD module.</li> <li>3. The quantum keys are distributed securely between User 1 and User 2 via the backend server using Socket.io and SSL/TLS.</li> <li>4. Eavesdropping detection and quantum key verification are performed.</li> <li>5. Keys are stored securely in MongoDB.</li> </ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"> <li>1. If eavesdropping is detected, the key exchange is aborted.</li> <li>2. If key verification fails, a new key exchange is initiated.</li> </ol>                                                                                                                                                                                                                                                            |
| <b>Exceptions</b>        | <ol style="list-style-type: none"> <li>1. Failure of the QKD module to generate or verify keys.</li> </ol>                                                                                                                                                                                                                                                                                                                                         |

Table 2.5: Use Case 5: Quantum Key Distribution (QKD)

|                          |                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-06                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Use Case Name</b>     | Send Group Encrypted Message                                                                                                                                                                                                                                                                                                                                                            |
| <b>Actors</b>            | User 1, Backend Server, Group Members                                                                                                                                                                                                                                                                                                                                                   |
| <b>Preconditions</b>     | The group has established a quantum secret key using QSS. User 1 is authenticated.                                                                                                                                                                                                                                                                                                      |
| <b>Description</b>       | User 1 sends a message to the group. The message is encrypted using AES-256 with a quantum secret key shared among group members using QSS. The backend server transmits the encrypted message securely to all group members.                                                                                                                                                           |
| <b>Postconditions</b>    | The message is transmitted to all group members securely, and can only be decrypted if a quorum of group members is present.                                                                                                                                                                                                                                                            |
| <b>Basic Flow</b>        | <ol style="list-style-type: none"><li>1. User 1 composes a group message in the React.js web interface.</li><li>2. The message is encrypted using AES-256 with a shared quantum key (QSS).</li><li>3. The backend server transmits the encrypted message securely to all group members.</li><li>4. Group members can only decrypt the message if a quorum of them is present.</li></ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"><li>1. If quorum is not met, the message cannot be decrypted.</li></ol>                                                                                                                                                                                                                                                                               |
| <b>Exceptions</b>        | <ol style="list-style-type: none"><li>1. Network failure during message transmission.</li></ol>                                                                                                                                                                                                                                                                                         |

Table 2.6: Use Case 6: Send Group Encrypted Message

|                          |                                                                                                                                                                                                                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-07                                                                                                                                                                                                                                                                                  |
| <b>Use Case Name</b>     | Receive Group Encrypted Message                                                                                                                                                                                                                                                        |
| <b>Actors</b>            | Group Members, Backend Server                                                                                                                                                                                                                                                          |
| <b>Preconditions</b>     | The group message has been sent by a user and encrypted using QSS. Group members must be authenticated and present to decrypt the message.                                                                                                                                             |
| <b>Description</b>       | Group members receive an encrypted message. They can only decrypt the message if a quorum of group members is present. The backend server manages message distribution and decryption.                                                                                                 |
| <b>Postconditions</b>    | The group message is successfully decrypted if the required number of group members are available.                                                                                                                                                                                     |
| <b>Basic Flow</b>        | <ol style="list-style-type: none"> <li>1. Group members receive a notification in the React.js interface.</li> <li>2. The system checks if quorum is met for decryption.</li> <li>3. If quorum is met, the message is decrypted using AES-256 and displayed to the members.</li> </ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"> <li>1. If quorum is not met, the decryption process is delayed.</li> </ol>                                                                                                                                                                          |
| <b>Exceptions</b>        | <ol style="list-style-type: none"> <li>1. Message tampering is detected.</li> </ol>                                                                                                                                                                                                    |

Table 2.7: Use Case 7: Receive Group Encrypted Message

## **2.2 Functional Requirements**

This section describes the functional requirements of the system expressed in natural language style. It is organized by system modules, each containing specific functional requirements.

### **2.2.1 Module 1: Quantum Key Distribution (QKD)**

The QKD module is responsible for generating and securely distributing cryptographic keys using the BB84 protocol.

1. The system shall implement the BB84 protocol to generate quantum encryption keys for secure key exchange between users.
2. The system shall verify quantum keys using quantum key verification techniques to detect potential eavesdropping.
3. The system shall securely distribute the generated quantum keys between the sender and the receiver without any exposure.
4. The system shall terminate the key exchange if an eavesdropping attempt is detected, and notify the admin for review.
5. The system shall store the generated keys temporarily for encryption purposes and erase them after message transmission is complete.

### **2.2.2 Module 2: AES Encryption**

The AES Encryption module handles the encryption and decryption of messages using AES-256 and the keys generated by the QKD module.

1. The system shall encrypt outgoing messages using AES-256 encryption with the quantum keys provided by the QKD module.
2. The system shall decrypt incoming messages using AES-256 with the quantum keys generated from QKD.
3. The system shall ensure that decrypted messages are only accessible by authorized, authenticated users.
4. The system shall log encryption and decryption processes for auditing purposes and provide reports to the admin.

5. The system shall re-encrypt stored messages with a new key after a specified period to maintain message security over time.

### **2.2.3 Module 3: Real-Time Communication**

This module facilitates secure real-time messaging between users over an encrypted channel using secure socket technology.

1. The system shall establish a secure communication channel using Socket.io for real-time message transmission.
2. The system shall support bi-directional encrypted messaging between authenticated users in real-time.
3. The system shall ensure message delivery through secure sockets using SSL/TLS for encrypted communication.
4. The system shall provide notification of message delivery, confirming that the recipient has received and decrypted the message.

### **2.2.4 Module 4: User Authentication**

The User Authentication module manages user access to the system.

1. The system shall authenticate users through a secure login mechanism using multi-factor authentication (MFA) for enhanced security.
2. The system shall validate user credentials and allow access to the chat interface upon successful authentication.
3. The system shall lock user accounts after multiple failed login attempts and notify the admin.
4. The system shall provide users with the ability to reset their password securely if forgotten.

### **2.2.5 Module 5: Front-end User Interface**

This module involves developing a responsive, user-friendly interface for users to interact with the Q-Sec system.

1. The system shall provide an intuitive, responsive web interface using React.js to allow users to send and receive encrypted messages.
2. The system shall display notifications and error messages clearly to the user regarding encryption status and message delivery.
3. The system shall ensure that users can initiate secure communication with a minimal number of interactions (e.g., sending a message within two clicks).

### **2.2.6 Module 6: Back-end Server**

The back-end server manages communication, key distribution, encryption processes, and overall system logic.

1. The system shall handle all incoming client requests, process message encryption, and manage key distribution securely.
2. The system shall log all system activity, including key exchanges, encryption, decryption, and real-time communication, for auditing purposes.
3. The system shall support scalability to handle up to 1000 concurrent users without performance degradation.
4. The system shall manage secure data storage for encrypted messages and user information using MongoDB.

### **2.2.7 Module 7: Quantum Secret Sharing (QSS)**

The QSS module is responsible for securely distributing a single quantum key among a group of users for group chats.

1. The system shall implement QSS to securely share a quantum key among group members.
2. The system shall ensure that encrypted messages sent to a group can only be decrypted if a quorum of group members is present.
3. The system shall support secure group messaging using QSS and AES-256 encryption.
4. The system shall manage the distribution of group messages securely through the backend using Socket.io and SSL/TLS encryption.

## 2.3 Non-Functional Requirements

This section specifies non-functional requirements. These quality requirements should be specific, quantitative, and verifiable.

### 2.3.1 Security

- The system shall use SSL/TLS to encrypt all communication between the client and the server to prevent unauthorized access.
- The system shall ensure that quantum keys and encrypted messages are never exposed to unauthorized users.
- The system shall detect and log any eavesdropping attempts and immediately notify the admin if an attempt is detected.
- The system shall rotate encryption keys regularly, with new keys generated for each session or after 24 hours.
- The system shall implement account lockout after 5 failed login attempts to protect against brute-force attacks.
- The system shall use QSS to ensure that a message can only be decrypted if the required number of group members are available.
- The system shall prevent unauthorized access to group messages by requiring group-based quantum key decryption.

### 2.3.2 Reliability

- The system shall have an uptime of 99.9
- The system shall automatically recover from any failure within 5 minutes and resume key distribution and message encryption.
- The system shall ensure message delivery integrity even in the case of a network disruption by queuing messages until reconnection.

### 2.3.3 Performance

- The system shall encrypt and transmit messages within 2 seconds for real-time communication.



- The system shall handle up to 1000 concurrent users with no more than a 5
- The system shall respond to user requests within 1 second, except during encryption processes.
- The system shall ensure that group messages are decrypted in real-time, provided that the quorum of group members is met.
- The system shall handle group communication with up to 50 members without performance degradation.

### **2.3.4 Usability**

- The system interface shall allow a user to send and receive messages with no more than 3 clicks.
- The system shall provide clear feedback to users regarding the status of encryption and message delivery.
- The system shall support both desktop and mobile browsers for ease of access.

## **2.4 Domain Model**

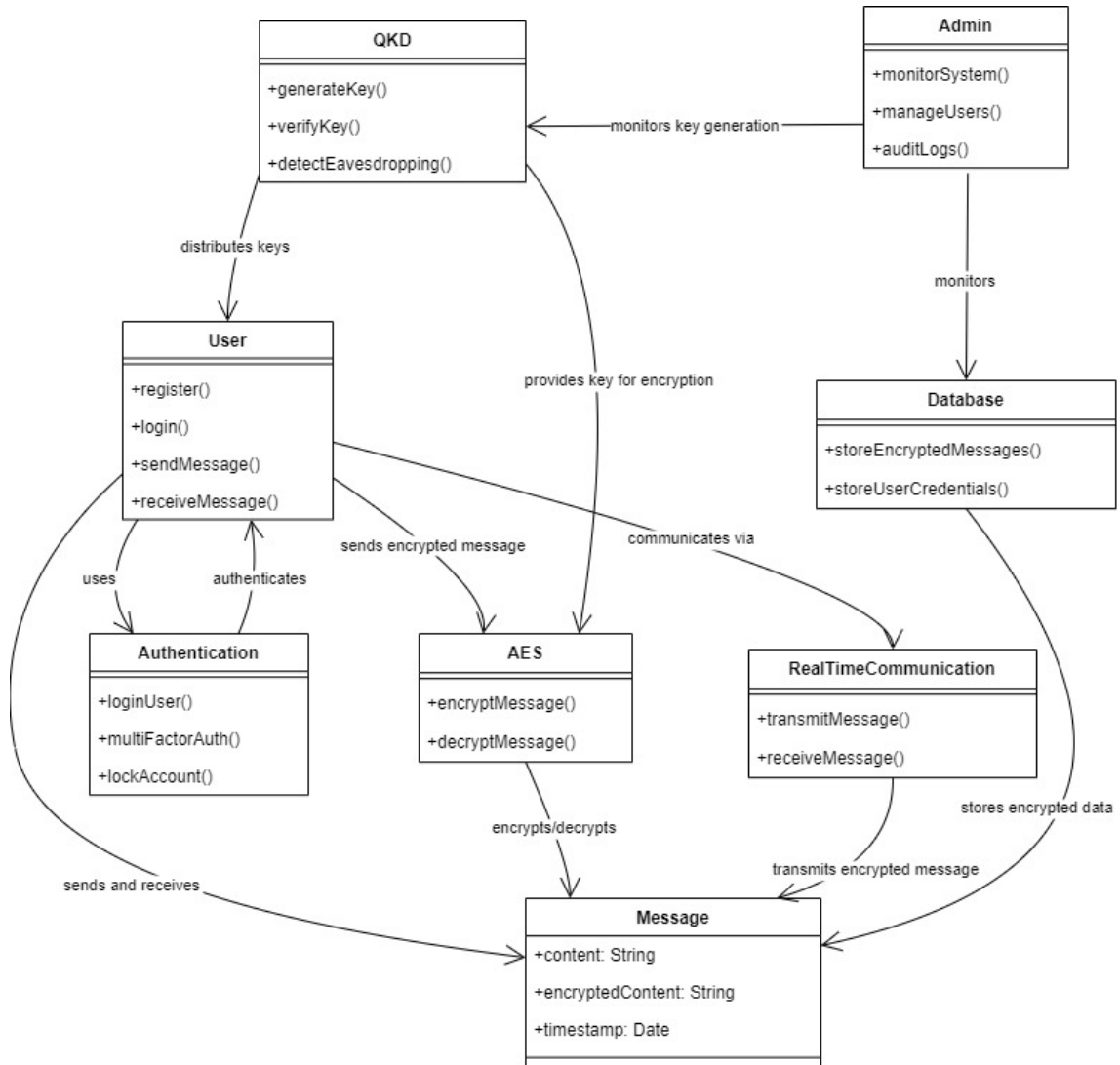


Figure 2.2: Domain Model Diagram for Q-Sec



# Chapter 3

## System Overview

The **Q-Sec** project aims to develop a quantum-secure chat application that combines cutting-edge quantum cryptography with industry-standard encryption methods to ensure secure and private communication. The system is designed to address the future threats posed by quantum computing on current cryptographic systems like RSA and ECC. With the advent of quantum computing, traditional cryptographic algorithms are vulnerable to being broken using quantum algorithms such as Shor's Algorithm. As a response, Q-Sec leverages Quantum Key Distribution (QKD) to exchange cryptographic keys securely, ensuring that communications are safe from quantum computing threats. Q-Sec also has Quantum Secret Sharing (QSS) to support secure communication in group chats. QSS allows a group of users to share a single quantum key, ensuring that messages sent within the group are securely encrypted and can only be decrypted when a quorum of users (a minimum number of group members) is present. This feature is especially useful for sensitive group communications where distributed trust is essential.

### 3.0.1 Functionality

The Q-Sec application provides a real-time, end-to-end encrypted chat system where messages are transmitted securely between users. The functionality of Q-Sec is built around the following key features:

1. **Quantum Key Distribution (QKD):** The BB84 protocol is used to securely exchange cryptographic keys between users. These keys are essential for encrypting and decrypting messages.
2. **AES-256 Encryption:** Once the quantum keys are securely exchanged, the system uses AES-256 encryption to encrypt messages sent between users. AES-256 is a symmetric encryption algorithm known for its strong security.

3. **User Authentication:** To ensure that only authorized individuals can access the system, Q-Sec integrates secure login systems with multi-factor authentication.
4. **Real-Time Communication:** Messages are exchanged in real-time through secure sockets using Socket.io. SSL/TLS encryption ensures that messages remain secure during transmission.
5. **Eavesdropping Detection:** The QKD module includes features to detect eavesdropping attempts, ensuring that any interception of the key exchange process is immediately flagged and stopped.
6. **Secure Message Storage:** Encrypted messages and user credentials are stored securely in MongoDB, with encryption applied to ensure data remains safe even when data is idle.

#### **3.0.2 Context and Design**

Q-Sec is designed to address the growing need for secure communication in an age where quantum computers are becoming increasingly capable of breaking traditional encryption methods. The application is built with a strong focus on security and usability, aiming to provide a seamless user experience while implementing complex encryption protocols behind the scenes.

#### **3.0.3 Key Modules of the System:**

1. **Quantum Key Distribution (QKD) Module:** Handles the secure exchange of encryption keys using the BB84 protocol. This module is crucial for ensuring that no one can intercept the keys during transmission, a key concern for quantum threats.
2. **AES Encryption Module:** Responsible for encrypting and decrypting messages using AES-256. The keys for this encryption are securely distributed through the QKD module.
3. **User Authentication Module:** Ensures only authorized users can access the chat system through secure login mechanisms, including multi-factor authentication for added security.
4. **Real-Time Communication Module:** This module ensures that messages are sent and received in real time. It uses Socket.io for fast, secure transmission, with SSL/TLS encryption ensuring data integrity during communication.

5. **User Interface (UI) Module:** The frontend of Q-Sec is designed to be intuitive and easy to use, with a web-based interface developed using React.js, HTML5, CSS3, and JavaScript. Users can easily send and receive encrypted messages without needing to understand the underlying cryptographic processes.

### **3.0.4 Background Information**

As quantum computing continues to advance, the threat of decryption of classical cryptographic algorithms like RSA and ECC increases. Shor's Algorithm, a quantum algorithm capable of factoring large numbers in polynomial time, has shown that quantum computers can easily break these traditional encryption methods. This creates a significant security risk for systems that rely on RSA or ECC for protecting sensitive data, particularly in sectors like healthcare, finance, government, and defense.

To address this growing concern, Q-Sec uses quantum cryptography for secure key exchange (QKD) combined with AES-256 encryption, which provides a robust defense against quantum-enabled attacks. By utilizing quantum technologies, Q-Sec ensures that the communication between users remains secure even as quantum computers become more powerful.

### **3.0.5 Design of the System**

The design of Q-Sec emphasizes simplicity, security, and scalability. The backend is powered by Node.js to handle system logic, manage client requests, and facilitate real-time messaging. MongoDB is used to store encrypted messages and user data, ensuring secure data management.

On the frontend, React.js is used to create a responsive, intuitive user interface that abstracts the complexity of quantum cryptography and encryption algorithms from the end user. This design ensures that users can engage with the system without requiring any understanding of the encryption mechanisms in place.

The QSS module manages the secure distribution of quantum keys for group chats. This ensures that encrypted messages can only be decrypted if the required number of group members are present. This module works alongside the QKD module and AES encryption to provide end-to-end encrypted communication for group chats.

## **3.1 Architectural Design**

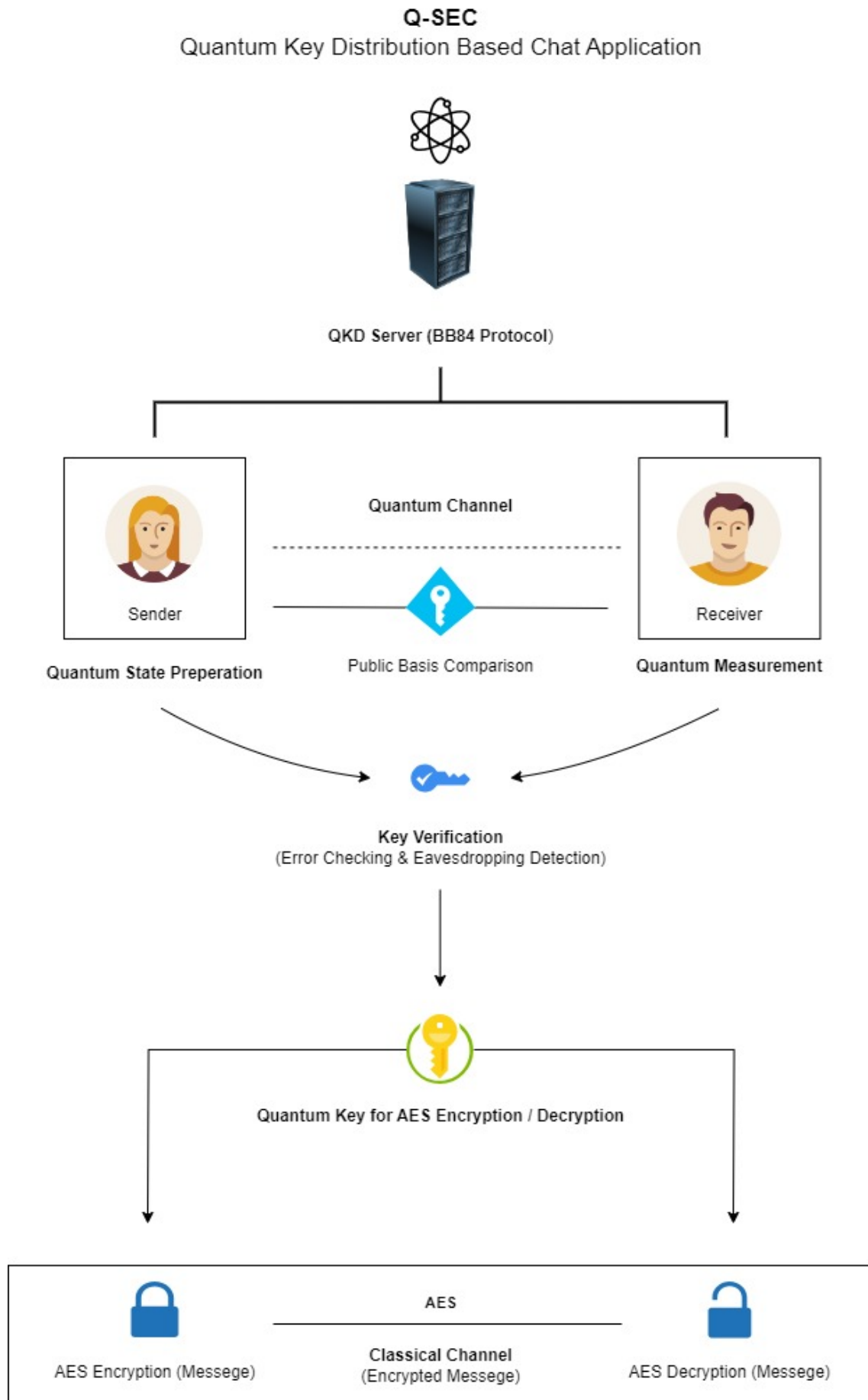


Figure 3.1: Architecture Diagram for Q-Sec

## 3.2 Design Models

### 3.2.1 Activity Diagram

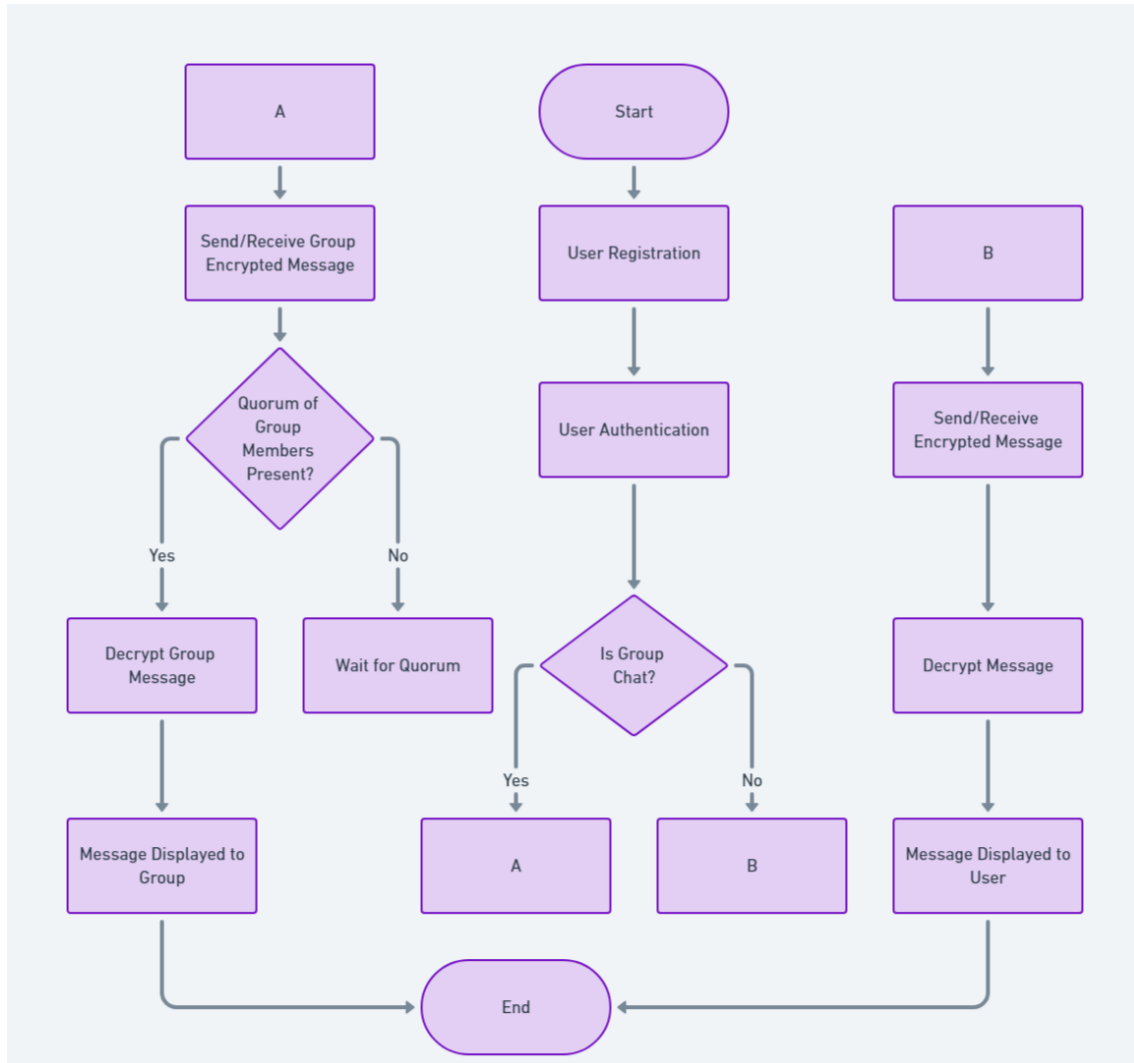


Figure 3.2: Activity Diagram for Q-Sec



3.2.2 Sequence Diagram

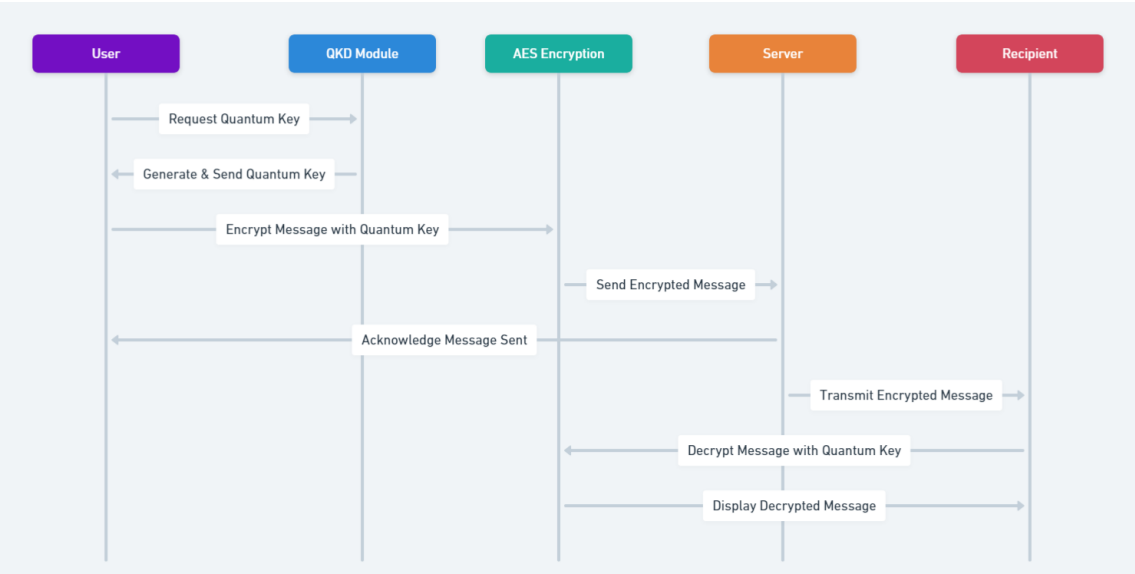


Figure 3.3: Sequence Diagram for Q-Sec

3.2.3 State Transition Diagram

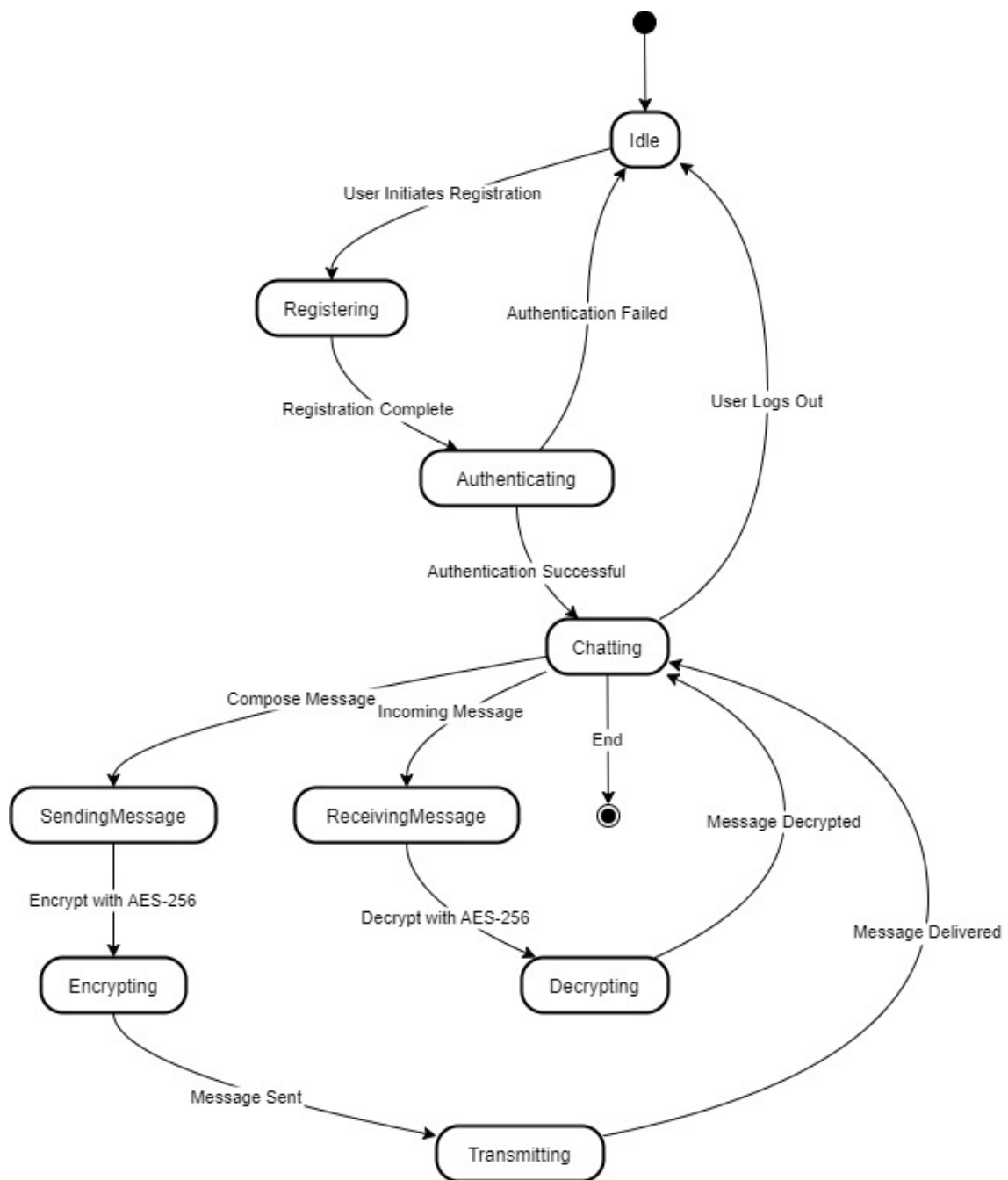


Figure 3.4: State Transition Diagram for Q-Sec

## 3.3 Data Design

This section explains how the information domain of the Q-Sec system is transformed into data structures. It describes how the major data entities are stored, processed, and organized. Additionally, it lists the databases or data storage used in the system.

### 3.3.1 Data Entities

The primary data entities in the Q-Sec system are Users, Messages, Quantum Keys, Encryption Keys, and System Logs. These entities are crucial for maintaining secure communication and managing the system. Below are the descriptions of each entity and their respective data structures:

#### 3.3.1.1 Users

**Attributes:**

- **username:** A unique string to identify the user.
- **password:** A securely hashed password for user authentication.
- **email:** The user's email address for account recovery and notifications.
- **role:** The user's role (e.g., Admin, End User).
- **status:** Represents the current status of the user account (e.g., active, locked).

**Storage:** MongoDB is used to store user data. User credentials are securely stored using hashing algorithms, and MongoDB ensures scalability and flexibility.

**Example Schema:**

```
{
  "_id": ObjectId("1234567890"),
  "username": "user1",
  "password": "hashed_password",
  "email": "user1@example.com",
  "role": "End User",
  "status": "active"
}
```

### 3.3.1.2 Messages

**Attributes:**

- `messageID`: A unique identifier for each message.
- `senderID`: A reference to the sender's username.
- `recipientID`: A reference to the recipient's username.
- `messageContent`: The encrypted content of the message.
- `timestamp`: The time the message was sent.
- `status`: The status of the message (e.g., delivered, read).

**Storage:** Messages are stored in MongoDB, and the content is encrypted using AES-256. The encrypted messages are stored in MongoDB securely.

**Example Schema:**

```
{
  "_id": ObjectId("9876543210"),
  "messageID": "msg12345",
  "senderID": "user1",
  "recipientID": "user2",
  "messageContent": "encrypted_message",
  "timestamp": "2024-10-01T10:00:00Z",
  "status": "delivered"
}
```

### 3.3.1.3 Quantum Keys

**Attributes:**

- `keyID`: A unique identifier for each quantum key.
- `keyValue`: The value of the quantum key used for encryption/decryption.
- `expiration`: The expiration time of the quantum key.
- `status`: Represents whether the key is active or expired.

**Storage:** Quantum keys are temporarily stored in MongoDB after generation and during the communication session.

**Example Schema:**

```
{
  "_id": ObjectId("1122334455"),
  "keyID": "key1",
  "keyValue": "quantum_key_value",
  "expiration": "2024-10-02T10:00:00Z",
  "status": "active"
}
```

#### 3.3.1.4 Encryption Keys

##### Attributes:

- **encryptionKeyID**: A unique identifier for each AES encryption key.
- **quantumKeyID**: The associated quantum key used to derive the AES key.
- **keyValue**: The AES-256 encryption key.
- **status**: Represents whether the key is active or expired.

**Storage:** Encryption keys are derived from the quantum keys and temporarily stored in MongoDB during communication sessions.

##### Example Schema:

```
{
  "_id": ObjectId("2233445566"),
  "encryptionKeyID": "aes_key1",
  "quantumKeyID": "key1",
  "keyValue": "aes_encryption_key_value",
  "status": "active"
}
```

#### 3.3.1.5 System Logs

##### Attributes:

- **logID**: A unique identifier for each log entry.
- **timestamp**: The time the log was generated.
- **logType**: The type of log (e.g., encryption error, key exchange failure).
- **message**: A description of the event.

- **status:** Represents the resolution status of the event (e.g., resolved, pending).

**Storage:** Logs are stored in MongoDB to allow system administrators to monitor and troubleshoot the application.

**Example Schema:**

```
{
  "_id": ObjectId("3344556677"),
  "logID": "log001",
  "timestamp": "2024-10-01T11:00:00Z",
  "logType": "error",
  "message": "Quantum key exchange failed",
  "status": "pending"
}
```

### 3.3.1.6 Groups

**Attributes:**

- **groupID:** A unique identifier for the group.
- **groupName:** The name of the group.
- **members:** A list of users in the group.
- **quantumSecretKey:** The quantum secret key shared among group members.

**Storage:** Group information and quantum secret keys are stored in MongoDB.

## 3.3.2 Data Flow and Processing

1. **User Registration and Authentication:** - User information is submitted via the React.js interface. - The backend (Node.js) validates the data and stores it securely in the MongoDB database. - Authentication is performed using multi-factor authentication (MFA) via Node.js backend.
2. **Message Transmission:** - Messages are composed by users and encrypted with AES-256 using a quantum key generated by the QKD module. - The backend securely transmits the encrypted messages to the recipient using Socket.io and SSL/TLS encryption.
3. **Quantum Key Generation and Distribution:** - The QKD module generates a quantum key using the BB84 protocol and securely distributes it between users. - The keys are validated for eavesdropping detection and used for AES encryption during the session.

4. **Real-Time Communication:** - Socket.io is used to transmit messages securely and in real-time. - The backend manages secure message transmission, ensuring low latency and secure sockets.

5. **System Logs:** - Logs are generated for each key exchange, encryption/decryption action, and system activity. - Admins monitor these logs to detect any security breaches or system issues.

#### 3.3.3 Data Storage Technologies

1. **MongoDB:** - MongoDB is the database used for storing users, messages, quantum keys, encryption keys, and logs. Its flexible schema is well-suited for handling dynamic data like user profiles, encrypted messages, and cryptographic keys.

2. **SSL/TLS Encryption:** - SSL/TLS protocols are used to secure communication channels, ensuring that data is encrypted in transit.

3. **AES-256 Encryption:** - AES-256 is used for encrypting both messages and keys, providing a high level of security.

#### 3.3.4 Conclusion

The data design for the Q-Sec system ensures the secure handling of sensitive data, including user credentials, encrypted messages, keys, and logs. MongoDB provides flexible storage for dynamic data, and encryption mechanisms like AES and QKD ensure the confidentiality of communications, both now and in the future.

# Bibliography

- [1] Van-Phuc Hoang Duc-Thuan Dam, Thai-Ha Tran. A survey of post-quantum cryptography: Start of a new race. *MDPI*, 2023. Accessed: 2024-09-27.
- [2] Kiran P. Sanjaikanth V. Analyzing the impact of quantum cryptography on network security. *ResearchGate*, 2023. Accessed: 2024-08-25.
- [3] Kannan A Santhosh Kumar Svn Shalini Subramani, Selvi M. Review of security methods based on classical cryptography and quantum cryptography. *Taylor Francis*, 2023. Accessed: 2024-08-26.
- [4] Luowen Qian William Kretschmer. Quantum cryptography in algorithmica. *ACM*, 2023. Accessed: 2024-08-26.